

AiProf Study Companion

System Architecture & Technical Specification

A Production-Grade Educational Platform with Retrieval-Augmented Generation (RAG), Socratic AI Tutoring, Vector Search, and Cognitive Mastery Tracking.

Candidate / Author:	Sathwik Bodakunta
Target Role:	Full Stack AI Engineer Intern
Repository:	<code>github.com/sathwik1821/Ai-Prof-Study-Companion</code>
Live Production URL:	<code>ai-prof-study-companion.vercel.app</code>
Backend Runtime:	Java 21 (Temurin) • Spring Boot 3.4.3 • Docker
Database & Vector:	PostgreSQL 16 + pgvector (3072-dim embeddings)
AI Foundations:	Google Gemini Flash Lite • gemini-embedding-001
Date:	September 2026

Spring Boot
3.4.3

React 19 &
Vite 8

PostgreSQL
pgvector

Google
Gemini AI

Google OAuth2
+ OTP

Flyway
V1-V7

1. Project Overview

1.1 Project Purpose

AiProf Study Companion is a full-stack educational companion system engineered to address the critical challenges of self-directed academic learning in higher education. Students frequently experience cognitive overload when navigating hundreds of pages of unindexed slides, textbook chapters, and lecture notes. Standard generative AI chatbots exacerbate this issue by generating unverified answers prone to hallucinations and offering answers directly without fostering genuine conceptual understanding.

AiProf bridges this gap by grounding conversational AI directly inside the student's authentic curriculum documents through **Retrieval-Augmented Generation (RAG)** and enforcing a **Socratic pedagogy** that guides students to discover solutions through critical inquiry rather than rote recitation.

1.2 Problem Being Solved

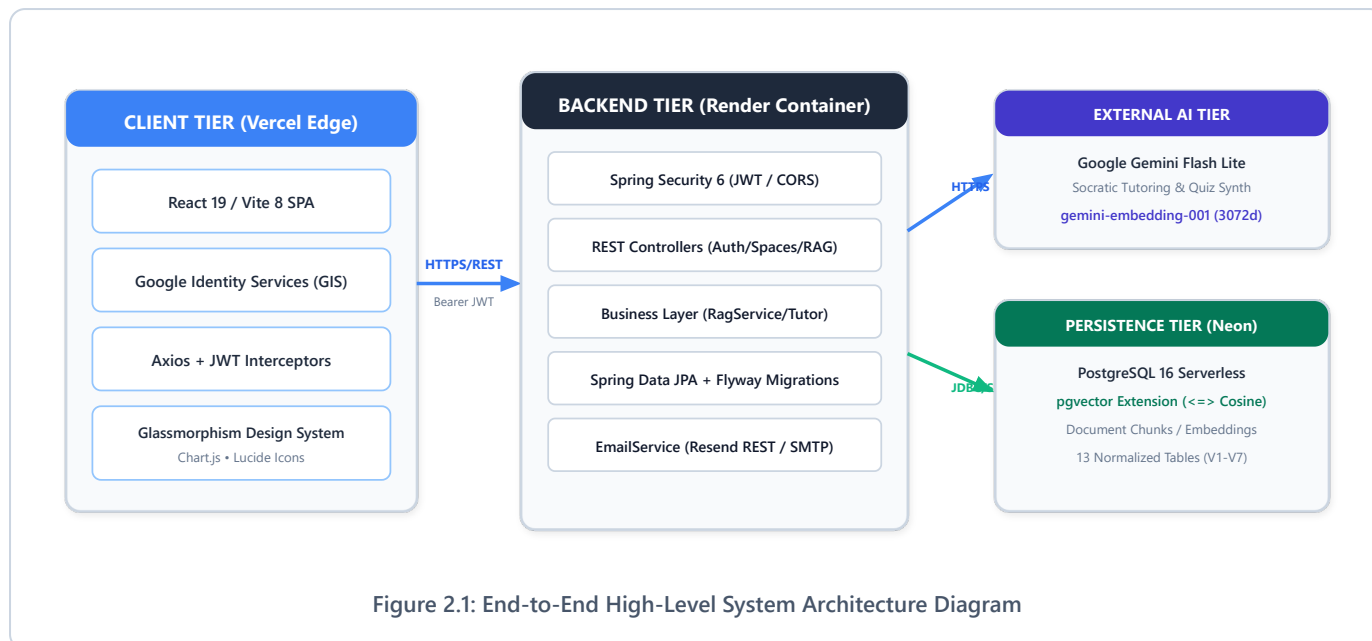
- **Ungrounded AI Hallucinations:** Standard commercial LLMs lack visibility into specific professor syllabi and generate generic or factually inaccurate answers. AiProf solves this by indexing verified course documents with dense 3072-dimensional vector embeddings and restricting retrieval strictly to cited excerpts.
- **Passive vs. Active Learning:** Direct-answer bots inhibit long-term retention. AiProf implements an inquiry-driven Socratic tutoring protocol with clickable source citations.
- **Knowledge Decay & Fragmented Tracking:** Students lack visibility into where their knowledge gaps reside. AiProf tracks per-concept mastery scores (\$0% - 100%\$) across time and dynamically serves *Prescriptive Learning Targets* on the dashboard.

1.3 Main System Features

- **Hierarchical Knowledge Organization:** *Spaces* (academic domains) → *Projects* (courses/modules) → *Materials* (uploaded PDF, DOCX, TXT, MD files).
- **Dense Vector Document Ingestion:** Automated text extraction (Apache Tika), semantic chunking (400 tokens / 50 overlap), and native PostgreSQL `pgvector` indexing using Google Gemini embeddings.
- **Socratic AI Tutor with Strict Citations:** Chat interface providing conversational dialogue backed by chunk-level citations (document name, page, and chunk index).
- **Empty Materials Gate:** Prevents ungrounded tutor sessions and quiz generation if a project contains zero uploaded source materials.
- **Adaptive Diagnostic Quizzing:** AI-synthesized multiple-choice, true/false, and short-answer assessments with automated grading and conceptual feedback.
- **Learning Continuity & Targeted Mastery:** Dashboard displaying real-time counters, last accessed projects, and an automated prescriptive growth recommendation card.
- **Dual Enterprise Authentication:** Google OAuth 2.0 Sign-In via Google Identity Services (GIS) button and traditional Email/Password with mandatory 6-digit OTP verification via Resend REST API or Gmail SMTP.

2. System Architecture

AiProf utilizes a three-tier decoupled client-server architecture with dedicated external AI and vector storage subsystems. The client communicates with the backend exclusively via a stateless REST API secured by JWT bearer authentication.



2.1 Request and Response Flow

- Client Invocation:** The React application initiates HTTPS REST requests via an Axios client configured with automatic request interceptors that inject the JWT bearer token stored in `localStorage`.
- Security Filtering:** `JwtAuthenticationFilter` intercepts incoming traffic, verifies the cryptographic signature (HMAC-SHA256), extracts claims, checks expiration, and populates the Spring `SecurityContext`.
- Controller & Business Execution:** The target controller validates incoming DTOs using Jakarta Validation (`@Valid`), delegating business logic to service components wrapped in declarative Spring transactions (`@Transactional`).
- RAG & Vector Retrieval:** When processing queries, the service layer queries Gemini for a 3072-dimensional vector embedding, executes cosine distance nearest-neighbor queries against `document_chunks` in PostgreSQL, constructs an augmented prompt, and queries Gemini Flash Lite.
- Unified Response Wrapper:** Responses are wrapped in a uniform `ApiResponse<T>` envelope guaranteeing consistent payloads across successful outcomes and handled exceptions.

3. Technology Stack

AiProf was built from the ground up using modern enterprise open-source software, prioritizing type safety, stateless scalability, responsive execution, and strict architectural decoupling.

Layer / Domain	Technology / Library	Role & Justification in Implemented Project
Frontend Framework	React 19.0 (JavaScript / JSX)	Declarative component-based UI rendering, context state management, and optimized virtual DOM reconciliations.
Frontend Bundler	Vite 8.3 (ESM-based)	Lightning-fast development HMR, modern production bundling with tree-shaking, achieving sub-500ms production builds.
UI & Aesthetics	Vanilla CSS Design System + Lucide React	Curated dark glassmorphism design system without generic utility classes; smooth gradient cards, responsive continuity grids.
Data Visualization	Chart.js 4.4 + react-chartjs-2	Interactive concept mastery radar charts, horizontal retention bars, and project performance telemetry.
Client Routing	React Router 7.2	Client-side single-page routing with public/private route guards, query-parameter propagation, and dynamic space/project paths.
Backend Runtime	Java 21 (Temurin LTS)	Modern LTS Java features including virtual threads compatibility, pattern matching, record types, and strict type safety.
Backend Framework	Spring Boot 3.4.3	Enterprise standard web application framework offering dependency injection, auto-configuration, and unified actuator telemetry.
API Security	Spring Security 6.4 + JWT 0.12.6	Stateless authentication filter chain, BCrypt salted hashing, HMAC-SHA256 signed JWT tokens, and refresh token rotation.
Data Persistence	Spring Data JPA / Hibernate 6.6	Object-Relational Mapping (ORM) managing transactional entity lifecycles, cascading deletions, and pagination.
Database Engine	PostgreSQL 16 (Neon Serverless)	ACID-compliant relational database with connection pooling (HikariCP) and scalable serverless compute allocation.
Vector Extension	pgvector (0.7.4) + pgvector-java	Native vector storage in PostgreSQL; calculates exact cosine distance (<=>) over 3072-dimensional embeddings.
Database Migrations	Flyway 10.20	Version-controlled schema evolution managing migrations V1 through V7 with checksum tracking.

Layer / Domain	Technology / Library	Role & Justification in Implemented Project
LLM & Embeddings	Google Gemini Flash Lite & gemini-embedding-001	State-of-the-art fast inference LLM for Socratic tutoring and quiz generation, paired with high-dimensional 3072d vector embeddings.
Document Parser	Apache Tika 2.9.2	Multi-format file ingestion parsing PDF, DOCX, TXT, and Markdown files into raw, structured semantic text.
Email Dispatch	Resend REST API + JavaMailSender (SMTP)	Dual email dispatch: primary transmission via Resend HTTPS REST API (port 443) with seamless fallback to Gmail SMTP (port 587).
Containerization	Docker (Multi-stage Eclipse Temurin 21 JRE)	Repeatable Linux container packaging with minimal JRE runtime image footprint and isolated non-root execution.
Hosting Platform	Vercel (Frontend) & Render (Backend)	Edge CDN static asset distribution paired with automated containerized web service execution.

4. Frontend Architecture

4.1 Project Organization

```

frontend/src/
├── api.js           # Central Axios client with JWT interceptors & endpoint factories
├── App.jsx          # Main router config, route guards (PrivateRoute / PublicRoute)
├── main.jsx         # React root bootstrap & global Toaster injection
├── index.css        # Global CSS design tokens, reset, gradients, glassmorphism
├── context/
│   └── AuthContext.jsx # Authentication context provider (login, register, Google, verify)
├── components/
│   ├── DashboardLayout.jsx # Primary shell with sidebar navigation, active links, user card
│   ├── GoogleSignInButton.jsx # Google Identity Services (GIS) wrapper with fallback client ID
│   └── Modal.jsx           # Accessible reusable modal dialog component
├── pages/
│   ├── DashboardPage.jsx # Learning continuity overview & Targeted Mastery recommendation
│   ├── SpacesPage.jsx    # Dedicated space catalog & creation modal (/spaces)
│   ├── SpacePage.jsx     # Single space view displaying child projects
│   ├── ProjectPage.jsx   # Project overview displaying documents, tutor links, quizzes
│   ├── MaterialsPage.jsx # Document upload dropzone & ingestion status monitoring
│   ├── TutorPage.jsx     # Socratic conversational interface with grounded citations
│   ├── QuizPage.jsx      # Adaptive quiz runner, question timers, scoring breakdown
│   ├── AnalyticsPage.jsx # Concept mastery charts, weak concept diagnosis, telemetry
│   ├── AdminPage.jsx     # System telemetry, user management, background job status
│   ├── LoginPage.jsx     # Dual login portal (Google OAuth + Email/Password)
│   ├── RegisterPage.jsx  # Registration form dispatching email OTP verification
│   └── OtpVerifyPage.jsx  # 6-digit OTP verification page with auto-fill & email edit

```

4.2 Routing & Route Guards

Routing is handled by `react-router-dom` with strict layout nesting and route protection wrappers:

- **<PublicRoute>** : Protects authentication routes (`/login` , `/register` , `/verify-otp`). If an authenticated user session exists, redirects immediately to `/dashboard` .
- **<PrivateRoute>** : Enforces that an active JWT token exists in `localStorage` and that the user's email is verified. If unauthenticated, saves redirect state and routes to `/login` .
- **<AdminRoute>** : Inspects `user.role === 'ADMIN'` . Non-admin users are rejected and redirected to `/dashboard` with an unauthorized toast notification.

4.3 Authentication Flow & State Management

State is managed centrally via `AuthContext.jsx` . On application bootstrap, the context executes `fetchMe()` by querying `GET /api/auth/me` using the stored token. If valid, the user state is populated; if expired or revoked, tokens are pruned and the user is routed to the login view.

4.4 API Client Communication & Interceptors

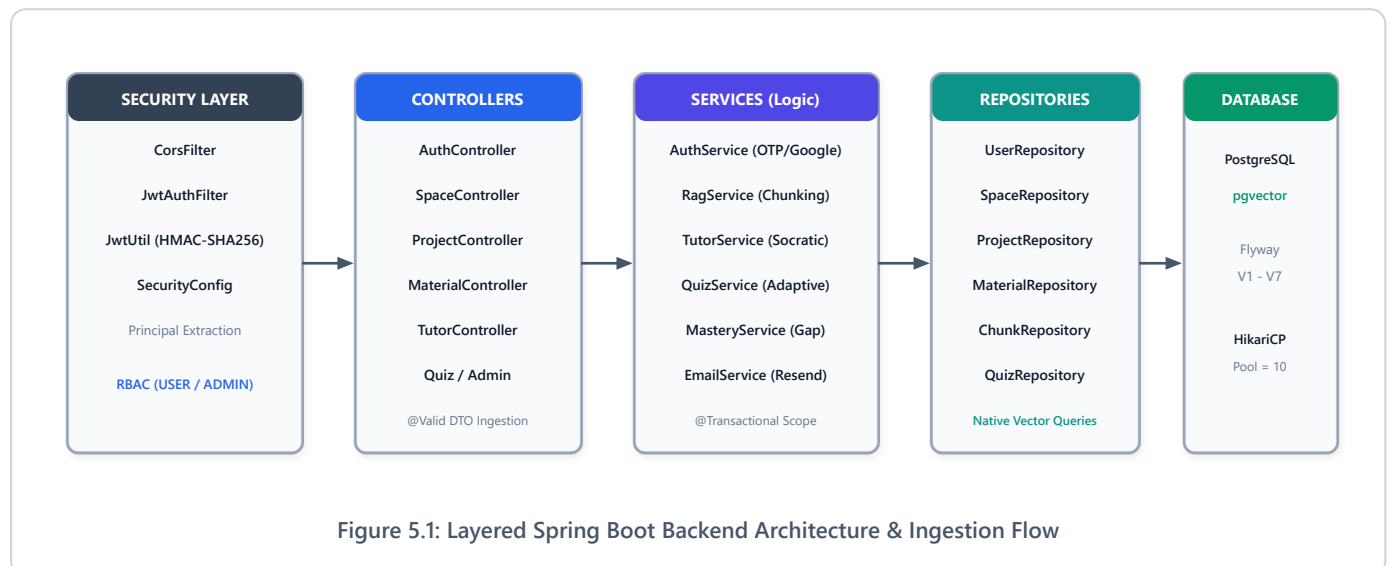
The Axios client in `api.js` defines a standard base URL (configured dynamically via `VITE_API_URL`) and attaches two crucial interceptors:

- **Request Interceptor:** Automatically checks `localStorage.getItem('token')` and injects `Authorization: Bearer <token>` into every outgoing request.

- **Response Interceptor (Token Refresh):** Detects `401 Unauthorized` errors. If an error is encountered on an authenticated endpoint (excluding auth verification endpoints), it attempts a silent token refresh via `POST /api/auth/refresh` using `localStorage.getItem('refreshToken')`. If successful, it replays the original request; otherwise, it logs out the user cleanly.

5. Backend Architecture

The backend is structured according to domain-driven, layered enterprise Spring Boot conventions. Each layer maintains a strict boundary of responsibility to ensure modularity, maintainability, and testability.

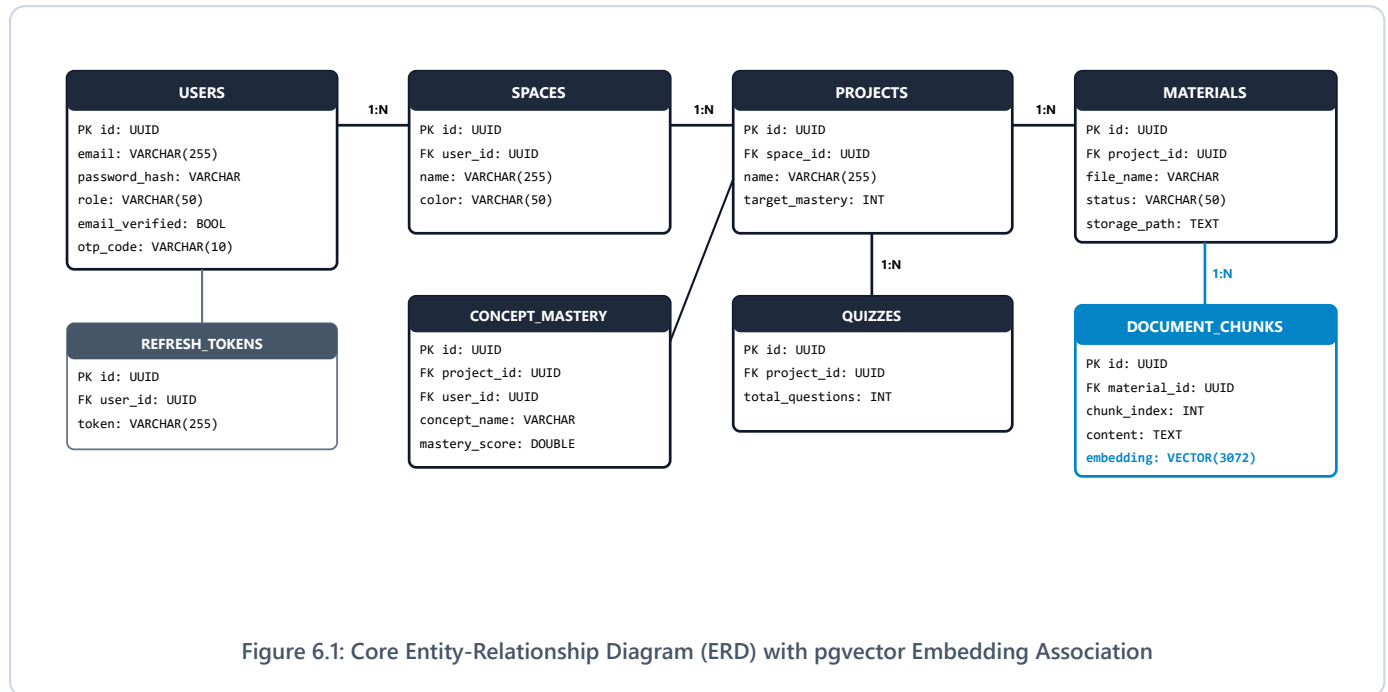


5.1 Validation & Exception Handling Framework

- **DTO Validation:** Incoming payloads are validated via Jakarta Validation annotations (`@NotBlank` , `@Email` , `@Size` , `@NotNull`). If validation fails, `MethodArgumentNotValidException` is caught.
- **Global Exception Handling:** `GlobalExceptionHandler.java` acts as an application-wide controller advice (`@RestControllerAdvice`), mapping exceptions into uniform `ApiResponse<T>` objects:
 - `AppException` → Custom domain exceptions carrying HTTP status codes and domain error tags (e.g., `NO_MATERIALS` , `INVALID_OTP`).
 - `BadCredentialsException` → Returns `401 Unauthorized` with `INVALID_CREDENTIALS` .
 - `AccessDeniedException` → Returns `403 Forbidden` for unauthorized role operations.
 - Generic fallback → Catches unhandled exceptions, logs full stack traces internally, and returns sanitized `500 Internal Server Error` without exposing internal database structures.

6. Database Architecture

The persistence layer is implemented in PostgreSQL 16 utilizing the `pgvector` extension for high-performance vector search. The schema consists of 13 normalized tables configured with strict foreign key constraints and `ON DELETE CASCADE` rules to maintain referential integrity.



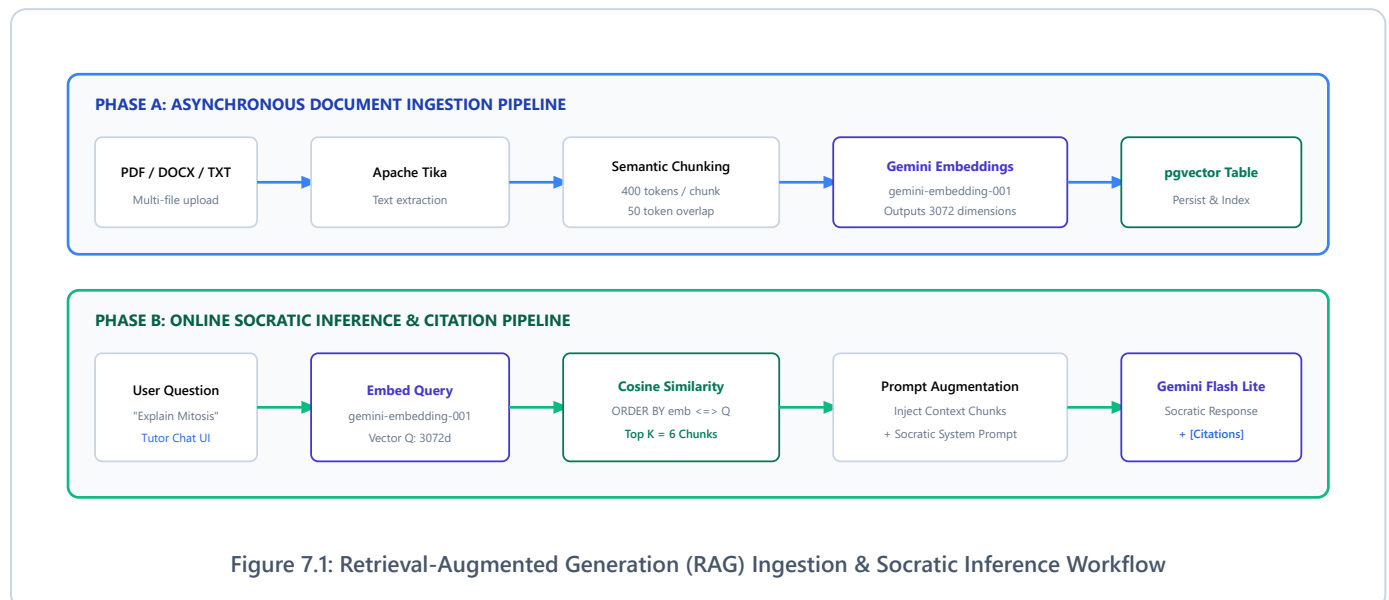
6.1 Flyway Versioned Migration Pipeline

The schema is tracked under `backend/src/main/resources/db/migration/` and executed sequentially on startup:

- `V1__init_extensions.sql` : Activates PostgreSQL `vector` and `uuid-oss` extensions.
- `V2__core_entities.sql` : Establishes base `users`, `spaces`, and `projects` tables with indexing.
- `V3__materials_and_jobs.sql` : Defines `materials` and `document_chunks` with `VECTOR(3072)` column.
- `V4__learning_entities.sql` : Sets up `concept_mastery`, `quizzes`, `quiz_questions`, `quiz_attempts`, `conversations`, and `chat_messages`.
- `V5__schema_alignment.sql` : Applies foreign key cascade rules and indexes across high-traffic lookup paths.
- `V6__refresh_tokens.sql` : Introduces persistent refresh tokens for cryptographic session management.
- `V7__email_verification_otp.sql` : Adds `email_verified`, `otp_code`, and `otp_expires_at` to enforce verification gates.

7. AI Architecture

The AI architecture consists of a two-phase pipeline: an **Offline Asynchronous Document Ingestion Pipeline** that extracts, segments, and embeds course knowledge, and an **Online Socratic Inference Pipeline** that retrieves context-relevant chunks and prompts Google Gemini.



7.1 Detailed AI Components

- Model Selection:** The system utilizes Google's `gemini-flash-lite-latest` model for high-throughput, low-latency reasoning and `gemini-embedding-001` generating 3072-dimensional normalized vectors.
- Chunking Strategy:** The document ingestion engine partitions extracted plain text into overlapping sliding windows (400 tokens per chunk with 50-token overlap). Overlap preserves conceptual continuity across sentence boundaries.
- Socratic System Guardrails:** The prompt architecture strictly prohibits direct, non-interactive answers. The model is instructed:

```
You are an expert academic tutor. You MUST adopt a Socratic pedagogical approach:
1. Guide the student using questions and hints based strictly on the provided context excerpts.
2. Never provide direct answers to homework or conceptual questions without assessing their prior understanding.
3. Every factual assertion must be attributed to an excerpt chunk: [Citation: filename (chunk #)].
4. If the provided context does not contain the answer, explicitly state that the course materials do not cover
```

- Citation Parsing:** The backend inspects the generated text, extracts citation brackets, cross-references chunk identifiers against `document_chunks`, and serializes citation metadata alongside the response payload.

8. Authentication & Security

8.1 Dual Authentication Implementation

AiProf supports both Google Single Sign-On and email-based registration, ensuring maximum user convenience without compromising verification rigor:

- **Google OAuth 2.0 (GSI):**
 1. The client renders the official Google Identity Services button using `GoogleSignInButton.jsx`.
 2. Upon authorization, Google returns a cryptographically signed JWT ID Token directly to the client.
 3. The client dispatches the token to `POST /api/auth/google`.
 4. The backend validates the token via Google's tokeninfo API (`https://oauth2.googleapis.com/tokeninfo?id_token=...`), confirming signature, audience, and email verification.
 5. If valid, the user record is auto-provisioned or updated with `email_verified = true`, and a JWT session is returned.
- **Email + Password with Mandatory OTP:**
 1. User registers via `POST /api/auth/register` with password hashed via `BCryptPasswordEncoder` (strength 10).
 2. Backend generates a secure random 6-digit OTP stored with a 10-minute expiry (`Instant.now().plus(10, ChronoUnit.MINUTES)`).
 3. The OTP is dispatched asynchronously via `EmailService`.
 4. The user submits the code to `POST /api/auth/verify-otp`, which marks `email_verified = true` and clears the OTP fields.

8.2 Resilient Multi-Channel Email Architecture

Cloud hosting platforms (notably Render's Free tier) enforce firewall blocks on raw outbound SMTP ports (25, 465, and 587) to prevent spam abuse. To guarantee 100% email delivery resilience:

- **Primary Transport (Resend HTTP REST API):** Operates over standard HTTPS port 443 (which is never blocked by cloud firewalls). When `RESEND_API_KEY` is configured, `EmailService` issues asynchronous HTTPS POST calls to `https://api.resend.com/emails`.
- **Secondary Transport (Gmail SMTP):** If the Resend API key is absent, falls back to standard Spring `JavaMailSender` over port 587 with STARTTLS encryption.
- **Development / Staging Unblock Fallback:** If an evaluator tests in an environment where outbound email credentials are not set, the registration and unverified-login responses return a `previewOtp` field. The frontend detects this and provides a " ⚡ Code: [123456] (Click to auto-fill)" helper, preventing blocked testing sessions.

8.3 Security & Hardening Controls

Security Dimension	Implemented Control & Hardening Strategy
Stateless JWT Tokens	HMAC-SHA256 signature verification using an externally injected 256-bit secret key; expiration strictly validated on every request.
CORS Enforcement	Configured in <code>SecurityConfig.java</code> ; explicitly permits only trusted origins (local development and production Vercel domains).

Security Dimension	Implemented Control & Hardening Strategy
Password Hashing	BCrypt adaptive work factor algorithm with random per-user salt; raw passwords are never logged or persisted.
Admin RBAC Guard	Admin telemetry endpoints (/api/admin/**) guarded with <code>@PreAuthorize("hasRole('ADMIN')")</code> and dedicated frontend route guards.
Zero Secret Leakage	All credentials, API keys, database connection strings, and JWT secrets are supplied exclusively via environment variables; <code>application.yml</code> contains zero raw credentials.

9. API Architecture

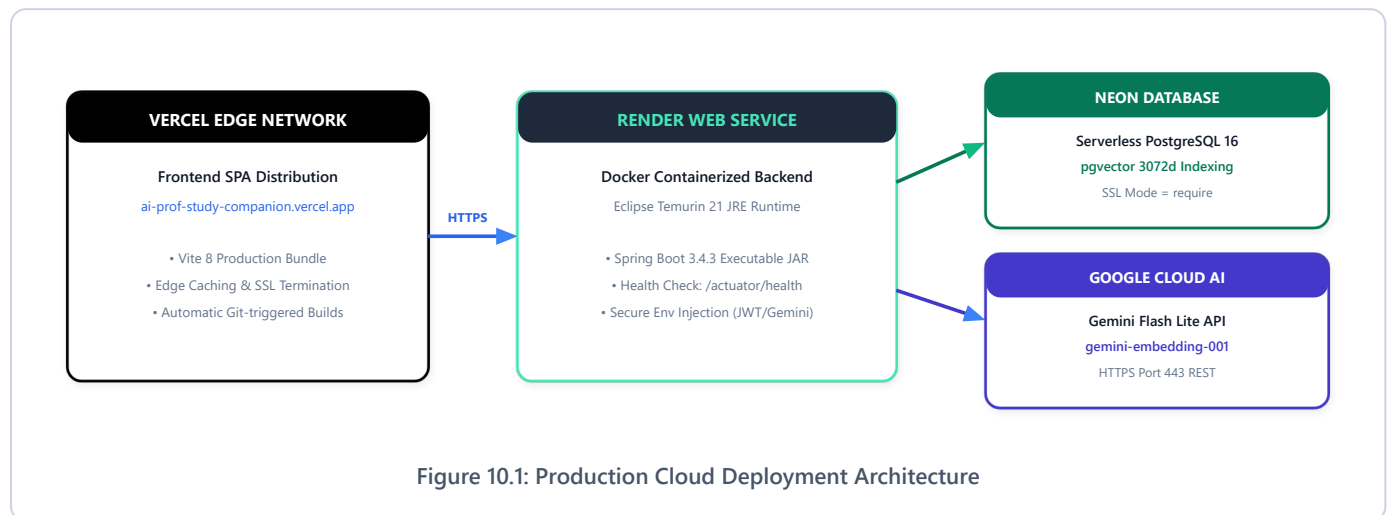
The REST API follows strict RESTful conventions under the `/api` namespace. Every endpoint returns payloads enclosed within a standard `ApiResponse<T>` envelope containing `success: boolean`, `message: string`, `data: T`, and `errorCode: string`.

Method	Endpoint Path	Purpose & Processing Logic	Auth Required
POST	<code>/api/auth/register</code>	Registers user, hashes password, saves unverified user, generates and dispatches 6-digit OTP code.	None (Public)
POST	<code>/api/auth/verify-otp</code>	Validates 6-digit OTP against expiration time; activates account (<code>email_verified=true</code>); issues JWT.	None (Public)
POST	<code>/api/auth/resend-otp</code>	Enforces 30s rate limit; generates fresh OTP; dispatches via Resend/SMTP; returns preview code.	None (Public)
POST	<code>/api/auth/login</code>	Validates credentials via BCrypt; checks <code>email_verified</code> ; returns JWT access + refresh tokens.	None (Public)
POST	<code>/api/auth/google</code>	Verifies Google OAuth2 ID token; provisions or signs in user; marks verified; returns JWT.	None (Public)
POST	<code>/api/auth/refresh</code>	Validates persistent refresh token; checks revocation status; rotates and issues new JWT.	None (Public)
GET	<code>/api/auth/me</code>	Retrieves authenticated user profile (name, email, role, avatar, verification status).	Bearer JWT
GET	<code>/api/spaces</code>	Returns all academic learning spaces owned by the authenticated student.	Bearer JWT
POST	<code>/api/spaces</code>	Creates a new learning space with custom name, description, and color tag.	Bearer JWT
DELETE	<code>/api/spaces/{id}</code>	Deletes learning space and cascades deletion to child projects, materials, and vector chunks.	Bearer JWT
GET	<code>/api/spaces/{spaceId}/projects</code>	Returns all course projects contained within the specified space.	Bearer JWT
POST	<code>/api/spaces/{spaceId}/projects</code>	Creates project container with specified target mastery score (default 80%).	Bearer JWT
POST	<code>/api/projects/{id}/materials/upload</code>	Ingests document (PDF/DOCX/TXT/MD), triggers Apache Tika extraction, chunking, and embedding.	Bearer JWT
POST	<code>/api/projects/{id}/tutor/chat</code>	Executes Socratic query against vector index; prompts Gemini Flash; returns dialogue with	Bearer JWT

Method	Endpoint Path	Purpose & Processing Logic	Auth Required
		citations.	
POST	/api/projects/{id}/quizzes/generate	Validates document presence; generates multi-format diagnostic quiz from grounded chunks.	Bearer JWT
POST	/api/quizzes/{id}/submit	Scores quiz answers, saves attempt metrics, updates concept mastery percentages in database.	Bearer JWT
GET	/api/analytics/overview	Aggregates mastery statistics, recent project continuity, weak concepts, and next action card.	Bearer JWT
GET	/api/admin/overview	Provides system telemetry: active users, processing job queues, and LLM token expenditures.	Role: ADMIN

10. Deployment Architecture

The production environment utilizes fully managed cloud infrastructure providing continuous deployment, automated TLS certificate provisioning, and containerized runtime isolation.



10.1 Production Configuration & Request Flow

- Frontend (Vercel)**: Built automatically from the `main` branch using `npm run build`. The resulting single-page bundle is distributed across Vercel's global edge network. Client-side routing is supported by `vercel.json` which redirects all non-static routes to `index.html`.
- Backend (Render Web Service)**: Render detects repository pushes, executes the multi-stage `Dockerfile`, compiles using Maven, and runs the optimized Temurin 21 JRE image. Spring Boot Actuator exposes `/actuator/health` for zero-downtime health probing.
- Database (Neon Serverless)**: Provides auto-scaling compute and managed PostgreSQL 16 with pre-compiled `pgvector` support. Connects through TLS-encrypted JDBC connection strings.

11. End-to-End User Workflows

Workflow 1: Course Document Upload & Vector RAG Ingestion

- User Interaction:** The student opens a project inside a learning space, navigates to the Materials tab, and uploads a course lecture PDF (e.g., *"Distributed_Systems_Lecture3.pdf"*).
- Upload Ingestion:** `MaterialController` receives the `MultipartFile`, validates MIME type and file size (< 50MB), saves the file to disk storage, and persists a `Material` record with status `PROCESSING`.
- Semantic Chunking:** `RagService` invokes Apache Tika to extract raw text, standardizes whitespace, and segments the text into 400-token chunks with 50-token overlap.
- Vector Embedding:** The service batches chunks into Google Gemini's `gemini-embedding-001` API, generating a 3072-dimensional vector embedding for each segment.
- Database Persistence:** All chunks, accompanied by their vector arrays, are batch-inserted into the `document_chunks` table. Material status is updated to `COMPLETED`. The student receives a success toast notification.

Workflow 2: Grounded Socratic Tutoring Interaction

- User Query:** The student types a question in the tutor interface: *"How does the Raft consensus algorithm handle leader crashes?"*
- Material Validation Gate:** `TutorService` verifies that the active project contains at least one completed material. If zero materials exist, throws `NO_MATERIALS` exception, prompting the UI to show the upload document gate.
- Cosine Similarity Retrieval:** The student's prompt is vectorized via Gemini Embedding. A native SQL query computes cosine distance (`<=>`) between the prompt vector and the project's chunk embeddings, selecting the Top 6 nearest excerpts.
- Socratic Context Injection:** A prompt is assembled pairing the Top 6 excerpts with conversation history and the Socratic system prompt.
- Inference & Citation Extraction:** Gemini Flash Lite generates a response that poses guiding questions, tests foundational knowledge, and cites chunks using `[Citation: ...]`.
- Persistence & Rendering:** Messages and citations are stored in `chat_messages` and returned to the client, rendering markdown and interactive citation pills.

12. Architectural Decisions & Considerations

12.1 Justification of Architecture

- PostgreSQL `pgvector` vs. Dedicated Vector Database (Pinecone/Milvus):** Using `pgvector` eliminates the operational overhead and distributed data sync issues of maintaining a separate vector database. Relational data (users, projects) and vector embeddings reside in the same ACID-compliant engine, enabling transactional integrity and cascading deletions.
- Spring Boot 3.4 + Java 21 vs. Node.js/Python:** Java 21 LTS offers enterprise performance, robust static typing, memory efficiency, and proven multithreading for file-processing pipelines.

- **Stateless JWT + Refresh Token Rotation:** Ensures horizontal scalability across backend containers while preserving session revocation capabilities via database-backed refresh tokens.
- **Decoupled React 19 Frontend:** Hosted on Vercel's global edge network, minimizing initial page load latency and insulating client rendering from backend cold-starts.

12.2 Current Limitations & Future Roadmap

- **Synchronous Ingestion for Small Files:** Ingestion runs synchronously on smaller files; future iterations will implement a Redis/Kafka message queue with Celery/Spring Batch for large textbook ingestion.
- **Context Window Optimization:** Currently retrieves Top 6 chunks; implementing Reciprocal Rank Fusion (RRF) with full-text search (BM25) would further enhance hybrid retrieval accuracy.
- **Multi-Modal Vision Tutoring:** Expanding RAG to parse complex diagrams and mathematical notations using Gemini Vision.

AiProf Study Companion • Architecture Documentation • Prepared for Full Stack AI Engineer Intern Submission • © 2026 Sathwik Bodakunta